

Cuaderno de Actividades de Aula: UT1 Manejo de Ficheros

Documento de Trabajo Síncrono Acompañado · 14 Retos Guiados Paso a Paso en Eclipse IDE

MÓDULO	UNIDAD DE TRABAJO	PROYECTO IDE	DOCENTE
Acceso a Datos (0486)	UT1: Manejo de Ficheros	UT01_ManejoFicheros	Sergio Capdevila Díez

Metodología de Trabajo en el Aula:

Cada vez que el profesor proyecte una diapositiva con el distintivo  **Práctica en Vivo en Aula: Actividad X.Y**, el alumnado abrirá su entorno de desarrollo (Eclipse / IntelliJ) y completará el reto práctico correspondiente en 10-15 minutos antes de avanzar a la siguiente sección teórica.



BLOQUE 1: Ficheros de Texto Plano (java.io y Java NIO.2)

Actividad 1.1: Diagnóstico de Charset y Conversión ISO-8859-1 TEXTO PLANO

a UTF-8

Momento de Aula: Tras
Diapositiva 03

Paquete:
act1_texto

Clase:
Actividad1_1_Charset.java

Problema Profesional: Migración de ficheros legados exportados por bases de datos antiguas en Windows (ISO-8859-1 o Windows-1252) hacia servidores web modernos en Linux (UTF-8), donde caracteres como tildes, eñes y diéresis se rompen visualmente (*Mojibake*).

 **DATOS DE PRUEBA (CREA DATOS/USUARIOS_LEGACY.TXT EN ISO-8859-1):**

```
1;García;Administración;garcia@empresa.com
2;Muñoz;Diseño Gráfico;muno@empresa.com
3;Peña;Logística y Almacén;pena@empresa.com
```

INSTRUCCIONES JAVA CLAVE:

INSTRUCCIÓN / MÉTODO	PARÁMETROS	UTILIDAD
<code>Files.newBufferedReader(...)</code>	<code>Path,</code> <code>StandardCharsets.ISO_8859_1</code>	Abre flujo de lectura decodificando los bytes en ISO-8859-1.
<code>Files.newBufferedWriter(...)</code>	<code>Path,</code> <code>StandardCharsets.UTF_8</code>	Abre flujo de escritura codificando en UTF-8 estándar.
<code>br.readLine()</code>	Ninguno	Lee una línea completa hasta <code>\n</code> . Devuelve <code>null</code> al final.
<code>bw.write(s); bw.newLine();</code>	<code>String</code>	Escribe la línea y añade el separador de salto de línea de la plataforma.

PROCEDIMIENTO PASO A PASO:

1. Crea la clase `Actividad1_1_Charset` en el paquete `act1_texto` .
2. Define las rutas: `Path origen = Path.of("datos", "usuarios_legacy.txt");` y `Path destino = Path.of("datos", "usuarios_utf8.txt");` .
3. Abre ambos flujos en un `try-with-resources` para garantizar el cierre seguro de archivos.
4. Implementa un bucle `while ((linea = br.readLine()) != null)` escribiendo cada línea con `bw.write(linea)` y `bw.newLine()` .
5. Muestra un mensaje por consola indicando el número de líneas migradas con éxito.

 **Alerta de Examen:** Olvidar `bw.newLine()` causará que todas las líneas se escriban pegadas en una sola línea continua sin saltos.

✓ **Resultado Esperado:** Al abrir `datos/usuarios_utf8.txt` en el editor, las palabras *Administración*, *Diseño* y *Logística* deben verse perfectas sin ningún símbolo extraño. Consola: ✓ Migración completada: 3 líneas convertidas a UTF-8.

Actividad 1.2: Filtrado y Normalización con `BufferedReader` y `BufferedWriter`

BUFFERS

Momento de Aula: Tras
Diapositiva 04

Paquete:
`act1_texto`

Clase:
`Actividad1_2_ConfigSanitizer.java`

Problema Profesional: Sanitizar ficheros de configuración `.ini` sucios antes de que una aplicación los cargue, eliminando líneas vacías, comentarios (`#` o `;`) y espacios en blanco, y normalizando las claves a mayúsculas.

 **DATOS DE PRUEBA (CREA `DATOS/CONFIG.INI`):**

```
# Configuración General del Servidor
; Parámetros de Red
servidor = 192.168.1.50
puerto = 8080

# Ajustes de Seguridad
ssl_enabled = true
timeout_segundos = 30
```

INSTRUCCIONES JAVA CLAVE:

MÉTODO	CLASE	UTILIDAD
<code>linea.trim()</code>	<code>String</code>	Elimina espacios en blanco iniciales y finales.
<code>linea.startsWith("#")</code>	<code>String</code>	Comprueba si la línea es un comentario.
<code>linea.split("=", 2)</code>	<code>String</code>	Divide la línea en exactamente dos partes: clave y valor.
<code>bw.newLine()</code>	<code>BufferedWriter</code>	Inserta un salto de línea nativo del sistema operativo.

PROCEDIMIENTO PASO A PASO:

1. Abre `datos/config.ini` con `BufferedReader` y `datos/config_clean.ini` con `BufferedWriter` .
2. En cada iteración, limpia con `linea = linea.trim()` .
3. Si `linea.isEmpty()` o empieza por `#` o `;` , sáltala con `continue` .
4. Divide por `=` limitando a 2 fragmentos: `String[] partes = linea.split("=", 2);` .
5. Escribe `partes[0].trim().toUpperCase() + "=" + partes[1].trim()` y añade `bw.newLine()` .

✓ **Resultado Esperado:** En `config_clean.ini` figurarán exactamente 4 líneas sin comentarios ni huecos:
`SERVIDOR=192.168.1.50` , `PUERTO=8080` , `SSL_ENABLED=true` y `TIMEOUT_SEGUNDOS=30` .

Actividad 1.3: Generador de Informe Formateado con PrintWriter FORMATO

Momento de Aula: Tras

Diapositiva 05

Paquete:

act1_texto

Clase:

Actividad1_3_FacturaPrintWriter.java

Problema Profesional: Generar un informe de facturación comercial en disco donde las descripciones de artículos se alineen a la izquierda y los importes monetarios a la derecha con dos decimales y el símbolo del euro, formando columnas perfectas.

INSTRUCCIONES JAVA CLAVE:

INSTRUCCIÓN / MÁSCARA	UTILIDAD
<code>pw.printf(formato, args...)</code>	Aplica máscaras de formateo idénticas a C/POSIX.
<code>%-10s</code>	Cadena de 10 caracteres alineada a la izquierda .
<code>%8.2f€</code>	Decimal de 8 caracteres con 2 decimales alineado a la derecha .
<code>%n</code>	Salto de línea portable e independiente de la plataforma.

PROCEDIMIENTO PASO A PASO:

1. Instancia `PrintWriter` sobre `datos/factura.txt` con `StandardCharsets.UTF_8`.
2. Imprime la cabecera del informe con líneas separadoras y títulos de columna.
3. Escribe 3 artículos usando `pw.printf` :
 - `PORT-01 | Portátil 15 Pulgadas | 750.50 | 2 uds`
 - `RAT-02 | Ratón Inalámbrico | 18.90 | 5 uds`
 - `MON-03 | Monitor 27 IPS | 199.99 | 1 ud`
4. Calcula la Base Imponible, el IVA (21%) y el Total Factura e imprímelos alineados al final.

 **Resultado Esperado:** Archivo `datos/factura.txt` generado con columnas perfectamente cuadradas.

Actividad 1.4: Operaciones Instantáneas con Java NIO.2 (Files)

NIO.2

Momento de Aula: Tras Diapositiva

Paquete:

Clase:

06

act1_texto

Actividad1_4_NIO2Files.java

Problema Profesional: Guardar y recuperar archivos de configuración pequeños o medianos en una sola línea de código, sin el boilerplate tradicional de abrir y cerrar flujos manualmente.

INSTRUCCIONES JAVA CLAVE:

MÉTODO DE FILES	RETORNO	UTILIDAD
<code>Files.writeString(Path, String)</code>	<code>Path</code>	Guarda todo el texto en disco en UTF-8 en una única instrucción.
<code>Files.readString(Path)</code>	<code>String</code>	Recupera todo el archivo en una sola variable <code>String</code> .
<code>Files.size(Path)</code>	<code>long</code>	Retorna el tamaño en bytes del archivo en disco.
<code>Files.exists(Path)</code>	<code>boolean</code>	Comprueba la existencia física del archivo.

PROCEDIMIENTO PASO A PASO:

1. Define un `String config = "db.host=localhost\ndb.port=3306\ndb.user=admin\ndb.pass=secret";`.
2. Escríbelo en `datos/database.properties` con `Files.writeString()`.
3. Comprueba con `Files.size()` e imprime los bytes ocupados.
4. Recupera el contenido íntegro con `Files.readString()` y muéstralo por pantalla.

📌 Actividad 1.5: Filtrado de Logs Masivos con Files.lines() (Stream Reactivo)

STREAMS

Momento de Aula: Tras
Diapositiva 07

Paquete:
act1_texto

Clase:
Actividad1_5_LogStreaming.java

Problema Profesional: Procesar un fichero de trazas de servidor de cientos de megabytes (`server.log`) extrayendo únicamente las incidencias críticas sin cargar todo el archivo en la memoria RAM del servidor.

🔧 INSTRUCCIONES JAVA CLAVE:

INSTRUCCIÓN	TIPO	UTILIDAD
<code>Files.lines(Path)</code>	<code>Stream<String></code>	Abre un stream que lee bajo demanda (lazy) línea a línea.
<code>.filter(l -> l.contains("[ERROR]"))</code>	Intermedia	Descarta líneas normales y retiene únicamente errores.
<code>.count()</code>	Terminal	Devuelve el número total de errores localizados.

⚠️ **Alerta de Examen:** Es **obligatorio** envolver `Files.lines()` en un bloque `try-with-resources` para liberar el descriptor del fichero del sistema operativo al terminar.

12
34

BLOQUE 2: Ficheros Binarios y Serialización de Objetos

Actividad 2.1: Telemetría de Sensores con DataOutputStream

BINARIO

Momento de Aula: Tras

Paquete:

Clase:

Diapositiva 09

act2_binarios

Actividad2_1_DataOutputStream.java

Problema Profesional: Almacenar telemetría de sensores industriales ahorrando el máximo ancho de banda y disco, persistiendo tipos numéricos nativos en binario estricto (enteros a 4 bytes, dobles a 8 bytes) en lugar de cadenas de caracteres.

INSTRUCCIONES JAVA CLAVE:

MÉTODO DE DATAOUTPUTSTREAM	BYTES OCUPADOS	UTILIDAD
<code>dos.writeInt(int)</code>	4 bytes	Escribe entero de 32 bits en binario big-endian.
<code>dos.writeUTF(String)</code>	2 + N bytes	Escribe 2 bytes con la longitud seguidos de caracteres UTF-8.
<code>dos.writeDouble(double)</code>	8 bytes	Escribe número de coma flotante de 64 bits (IEEE 754).
<code>dos.writeBoolean(boolean)</code>	1 byte	Escribe <code>0x01</code> para <code>true</code> o <code>0x00</code> para <code>false</code> .

PROCEDIMIENTO PASO A PASO:

1. Instancia `DataOutputStream` sobre `BufferedOutputStream` y `FileOutputStream("datos/telemetry.dat")`.
2. Escribe 3 sensores con la siguiente secuencia exacta:
 - Sensor 101: `101` (int), `"Caldera Central"` (String), `84.5` (double), `false` (boolean).
 - Sensor 102: `102` (int), `"Turbina 01"` (String), `102.3` (double), `true` (boolean).
 - Sensor 103: `103` (int), `"Compresor Gas"` (String), `45.0` (double), `false` (boolean).
3. Cierra el flujo y comprueba en el explorador que el archivo ocupa menos de 75 bytes y es ilegible en un bloc de notas.

📌 Actividad 2.2: Detección Limpia de Fin de Archivo con

EOF CONTROL

EOFException

Momento de Aula: Tras

Paquete:

act2_binarios

Clase:

Actividad2_2_DataInputStreamEOF.java

Diapositiva 10

Problema Profesional: Leer secuencialmente un archivo binario hasta el final con `DataInputStream`. Al leer números primitivos, un `-1` puede ser un dato real válido (temperatura negativa), por lo que Java no retorna `-1` ni `null`, sino que lanza obligatoriamente `EOFException`.

🔧 INSTRUCCIONES JAVA CLAVE:

```
try (DataInputStream dis = new DataInputStream(new BufferedInputStream(new FileInputStream(f))
    while (true) {
        int id = dis.readInt();
        String nom = dis.readUTF();
        double temp = dis.readDouble();
        boolean alr = dis.readBoolean();
        System.out.printf("[%d] %s: %.2f°C (Alarma: %b)%n", id, nom, temp, alr);
    }
} catch (EOFException e) {
    System.out.println("✅ Fin de fichero alcanzado limpiamente.");
}
```

⚠️ **Alerta de Examen:** Debes leer los campos en el **mismo orden exacto** en que fueron escritos. Si intentas leer `readDouble()` donde se escribió un `int`, los bytes se desfazarán por completo corrompiendo la lectura.

Actividad 2.3: Modificación Directa con RandomAccessFile (RAF)

RAF

Momento de Aula: Tras Diapositivas

Paquete:

act2_binarios

Clase:

Actividad2_3_RandomAccessFile.java

11 y 12

Problema Profesional: Actualizar directamente en el disco físico el salario de un empleado concreto de una base de datos sin leer ni duplicar los demás registros.

MATEMÁTICAS DEL REGISTRO FIJO (36 BYTES):

id (4B) + apellido (10 chars con writeChars = 20B) + departamento (4B) + salario (8B) = **36 Bytes**.

- Inicio del registro `i`: `offset_base = i * 36`
- Ubicación del salario en registro `i`: `offset_salario = (i * 36) + 28` (4 + 20 + 4)

INSTRUCCIONES JAVA CLAVE:

MÉTODO	UTILIDAD
<code>raf.seek(long pos)</code>	Posiciona el cabezal en el byte absoluto <code>pos</code> .
<code>raf.writeDouble(nuevoSalario)</code>	Sobrescribe los 8 bytes del salario en disco.
<code>raf.getFilePointer()</code>	Devuelve la posición actual del cursor.
<code>raf.length()</code>	Devuelve el tamaño total en bytes del fichero.

PROCEDIMIENTO PASO A PASO:

1. Crea el archivo `datos/empleados.dat` con 3 empleados.
2. Implementa `modificarSalario(int posicion, double nuevoSalario)`:
 - Calcula el offset exacto: `(posicion * 36) + 28`.
 - Abre con `new RandomAccessFile(f, "rw")`.
 - Ejecuta `raf.seek(offset)` y `raf.writeDouble(nuevoSalario)`.
3. Vuelve a leer el registro situando el cursor en `posicion * 36` y verifica que el salario se actualizó sin tocar el resto de datos.



Actividad 2.4: Serialización de Objetos con Campos Transient

SERIALIZACIÓN

Momento de Aula:

Tras Diapositiva 13

Paquete:

act2_binarios

Clases: Usuario.java ,

Actividad2_4_SerializacionTransient.java

Problema Profesional: Guardar el estado completo de objetos en disco respetando la legalidad (RGPD), evitando que campos sensibles de memoria RAM (tokens de autenticación, contraseñas) se serialicen en el fichero físico.



INSTRUCCIONES JAVA CLAVE:

```
public class Usuario implements Serializable {  
    private static final long serialVersionUID = 1L;  
    private int id;  
    private String username;  
    private transient String sessionToken; // ¡NO se guardará en disco!  
    // Constructor, getters y toString...  
}
```



PROCEDIMIENTO PASO A PASO:

1. Crea la clase `Usuario` con el modificador `transient String sessionToken` .
2. Instancia dos usuarios con tokens activos y serialízalos en `datos/usuarios.ser` con `ObjectOutputStream.writeObject()` .
3. Deserialízalos con `ObjectInputStream.readObject()` y constata que `sessionToken` toma su valor por defecto: `null` .

📌 Actividad 2.5: El Gran Reto - Persistencia Incremental

MIOBJECTOUTPUTSTREAM

sin StreamCorruptedException

Momento de Aula:

Tras Diapositivas 14 y
15

Paquete:

act2_binarios

Clase:

Actividad2_5_AppendObjetos.java

Problema Profesional: Añadir objetos a un fichero serializado existente con `FileOutputStream(f, true)` provoca el colapso del stream con `StreamCorruptedException: invalid type code: AC`, debido a que el constructor de `ObjectOutputStream` inyecta una cabecera de 4 bytes intermedia en mitad del archivo.

🔧 SOLUCIÓN DOCENTE CANÓNICA:

```
// 1. Subclase que anula la escritura de cabecera intermedia
class MiObjectOutputStream extends ObjectOutputStream {
    public MiObjectOutputStream(OutputStream out) throws IOException { super(out); }
    @Override
    protected void writeStreamHeader() throws IOException {
        // Al dejar este método vacío, NO se escribe cabecera secundaria
    }
}

// 2. Patrón de apertura según estado del fichero
boolean yaExisteConDatos = f.exists() && f.length() > 0;
try (FileOutputStream fos = new FileOutputStream(f, true);
    ObjectOutputStream oos = yaExisteConDatos
        ? new MiObjectOutputStream(fos)
        : new ObjectOutputStream(fos)) {
    oos.writeObject(nuevoUsuario);
}
```

✅ **Resultado Esperado:** Al ejecutar el programa 3 veces y leer todos los objetos con un bucle `while(true)` hasta `EOFException`, se recuperan todos los registros limpiamente sin ninguna excepción de corrupción.



BLOQUE 3: Ficheros XML (DOM, SAX, StAX y JAXB)

Actividad 3.1: Consulta y Modificación de Catálogo con DOM

DOM

Momento de Aula: Tras Diapositiva

17

Paquete:

act3_xml

Clase:

Actividad3_1_DOMModificacion.java

Problema Profesional: Cargar un documento XML en memoria RAM para inspeccionar elementos y modificar precios, evitando la trampa habitual de confundir los saltos de línea con etiquetas hijas (nodos `#text`).

 **DATOS DE PRUEBA (CREA DATOS/CATALOGO.XML):**

```
<?xml version="1.0" encoding="UTF-8"?>
<catalogo>
  <producto id="P101">
    <nombre>Disco SSD 1TB</nombre>
    <precio>79.99</precio>
    <stock>15</stock>
  </producto>
  <producto id="P102">
    <nombre>Memoria RAM 16GB</nombre>
    <precio>45.50</precio>
    <stock>30</stock>
  </producto>
</catalogo>
```

INSTRUCCIONES JAVA CLAVE:

1. `Document doc = DocumentBuilderFactory.newInstance().newDocumentBuilder().parse(f);`
2. `doc.getDocumentElement().normalize();` (¡Obligatorio para unir nodos de texto adyacentes!).
3. Recorrer `getElementsByTagName("producto")` verificando `if (nodo.getNodeType() == Node.ELEMENT_NODE)`.
4. Extraer el precio, multiplicarlo por `1.10` (+10% inflación) y actualizarlo con `setTextContent()`.



Actividad 3.2: Generación de XML y Volcado Formateado con

TRANSFORMER

Transformer

Momento de Aula: Tras

Paquete:

Diapositiva 18

act3_xml

Clase:

Actividad3_2_DOMTransformer.java

Problema Profesional: Construir un árbol XML desde código Java y exportarlo a disco con sangría limpia (*pretty-print*) para intercambio de información entre sistemas externos.



CONFIGURACIÓN INDENTADA DE TRANSFORMER:

```
Transformer tf = TransformerFactory.newInstance().newTransformer();
tf.setOutputProperty(OutputKeys.INDENT, "yes");
tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");
tf.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
tf.transform(new DOMSource(doc), new StreamResult(new File("datos/alumnos.xml")));
```



Actividad 3.3: Analizador Reactivo sin Memoria con SAX

SAX PUSH

Momento de Aula: Tras

Paquete:

Diapositiva 19

act3_xml

Clase:

Actividad3_3_SAXAnalizador.java

Problema Profesional: Calcular el valor total del inventario de un catálogo XML masivo (varios gigabytes) con consumo de memoria RAM cercano a 0 MB utilizando el procesador reactivo basado en eventos `DefaultHandler`.



CICLO DE VIDA DEL HANDLER:

- `startElement()` : Limpiar acumulador con `buffer.setLength(0)`.
- `characters(ch, start, len)` : Acumular con `buffer.append(ch, start, len)` (¡el texto puede venir en trozos!).
- `endElement()` : Si la etiqueta es "precio", parsear a double y sumar.
- `endDocument()` : Calcular y mostrar la media de precios final.

Actividad 3.4: Mapeo Declarativo Objeto-XML con JAXB

JAXB

Momento de Aula: Tras
Diapositiva 20

Paquete:
act3_xml

Clases: Curso.java ,
Actividad3_4_JAXBMapeo.java

Problema Profesional: Persistir y recuperar entidades complejas en XML mediante anotaciones declarativas (Jakarta XML Binding) sin manipular nodos DOM manualmente.

CÓDIGO ESENCIAL:

```
@XmlRootElement(name = "curso")
@XmlAccessorType(XmlAccessType.FIELD)
public class Curso {
    @XmlAttribute private String codigo;
    @XmlElement private String nombre;
    @XmlElement private int horas;
    public Curso() {} // ¡Constructor vacío obligatorio para JAXB!
}

// Marshaller (Java -> XML)
JAXBContext ctx = JAXBContext.newInstance(Curso.class);
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(curso, new File("datos/curso.xml"));

// Unmarshaller (XML -> Java)
Curso c = (Curso) ctx.createUnmarshaller().unmarshal(new File("datos/curso.xml"));
```